IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Árboles en cadena que corrigen errores

Teoría, práctica y, al final, material de consulta

Sprint 8 · S08_03 · teoría breve, 25 min de práctica y material de consulta

LA IDEA

Cada árbol repasa los fallos del anterior

Como los simulacros de un examen: tras cada uno repasas **solo lo que has fallado**. Ejemplo con 6 reservas y 3 árboles de una sola pregunta (predecimos el precio por noche):

reserva	suite (1 = sí)	alta (1 = sí)	real (€)	media	tras árbol 1	tras árbol 2	tras árbol 3
A	0	0	60	100	87,5	81,25	76,25
B	0	1	80	100	87,5	81,25	86,25
C	1	0	140	100	125	137,5	132,5
D	1	1	160	100	125	137,5	142,5
E	0	0	70	100	87,5	81,25	76,25
F	0	1	90	100	87,5	81,25	86,25
error medio (€)				33,33	17,50	11,25	9,58

Árbol 1: mira **suite**, suma +50 a las suites y −25 al resto, pero con **learning_rate=0,5** aplica **la mitad**.

BOSQUE FRENTE A BOOSTING · Y EL PASO

Dos formas de juntar árboles

	Bosque (bagging)	Boosting
Árboles	Grandes, independientes, en paralelo	Pequeños, en cadena: cada uno corrige al anterior
Reduce sobre todo	Varianza (el modelo cambia mucho si cambian los datos)	Sesgo (el modelo es demasiado simple y falla siempre en lo mismo)
Riesgo	Poco sobreajuste	**Sobreajusta si se añaden demasiados árboles**

learning_rate	error medio tras 3 árboles
0,2	17,80
0,5	9,58
1,0	3,33

El paso: pequeño = más árboles

Un paso pequeño avanza más despacio y necesita **más árboles**, pero afina más. Con datos reales y ruido, un paso grande sobreajusta antes.

SOBREAJUSTE

Demasiados árboles memorizan

En entrenamiento

El AUC **sube siempre**: cada árbol nuevo corrige algo más de lo que el modelo ya vio.

En validación

El AUC sube hasta un **máximo** y después **baja**: el modelo empieza a aprender ruido.

La separación entre las dos curvas es el sobreajuste: el boosting sobreajusta si se añaden demasiados árboles.

Por eso hay que parar en el mejor punto, y ese punto se busca **con datos de validación**, no con el test.

PARADA TEMPRANA

Para cuando el simulacro deja de mejorar

Entrenar

Con estas reservas se **construyen** los árboles.

Validar

Decide **cuándo parar**.

Test

No se toca hasta el final.

Ronda	AUC validación	Mejor hasta ahora	Rondas sin mejorar
6	0,740	0,741	1
7	0,742	0,742	0
8	0,741	0,742	1
9	0,739	0,742	2
10	0,738	0,742	3 · **¡para!**

Paciencia de 3 rondas (la ronda 5 llegó a 0,741): se detiene y **se queda con la ronda 7**. Con datos reales, una paciencia mayor (30).

XGBOOST

Un máximo alto y que pare solo

Hiperparámetro	Qué controla
n_estimators	Máximo de árboles (con parada, se pone alto)
learning_rate	Paso de cada árbol (menor = más fino)
max_depth	Profundidad de cada árbol (2 a 6 suele bastar)
subsample · **colsample_bytree**	Fracción de filas / columnas por árbol
reg_lambda · **min_child_weight**	Regularización y tamaño mínimo de nodo
early_stopping_rounds	Rondas sin mejorar para detenerse

Con **parada temprana**, **n_estimators** es solo un tope: el entrenamiento se detiene cuando la validación deja de mejorar.

LIGHTGBM

Crece por hojas: manda num_leaves

XGBoost: por niveles

max_depth=3 permite como mucho 2^3 = **8 hojas**, nivel a nivel.

LightGBM: por hojas

num_leaves=8 fija las **8 hojas**; crece siempre por la hoja que más mejora. **min_child_samples** evita hojas diminutas.

Misma idea, otro modo de crecer: se limita el nº de hojas, no los niveles.

COMPARAR IMPLEMENTACIONES

La implementación importa menos que los datos

AUC

Las tres implementaciones suelen dar un AUC **muy parecido**.

Velocidad

LightGBM y XGBoost **escalan mucho mejor** que la versión de scikit-learn con muchos datos.

Nulos y categóricas

LightGBM y XGBoost los admiten de forma **nativa**; scikit-learn pide preprocesarlos.

Decide el coste: tiempo, mantenimiento y explicabilidad. Las importancias describen el modelo, no las causas.

PRÁCTICA · CUADERNO S08_03

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación y ejemplo	Reparto entrenar/validar/test y AUC según el número de árboles	Clase
Ejercicio 1	El mejor número de árboles y cuánto cae el AUC	Clase
Ejercicio 2	XGBoost con parada temprana	Clase
Ejercicio 3	LightGBM con parada temprana	Casa
Ejemplo	Importancia de variables de XGBoost	Casa
Mini-reto	Comparar scikit-learn, XGBoost y LightGBM	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el ✔**. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

Cómo está ordenado

En el **mismo orden que el cuaderno**: ejercicios 1 a 3, importancia y mini-reto. Cada uno trae su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
1	Mejor nº de árboles	11 y 12
2	XGBoost	13 y 14
3 (casa)	LightGBM	15 y 16
Ejemplo (casa)	Importancia de variables	17 y 18
Mini-reto (casa)	Tres implementaciones	19 y 20

CONSULTA · EJERCICIO 1

El mejor número de árboles (ejercicio 1)

```
gb = GradientBoostingClassifier(n_estimators=300, learning_rate=0.2, max_depth=3, random_state=SEMILLA).fit(Xfit, yfit)
auc_val_por_arbol = [roc_auc_score(yval, p[:, 1]) for p in gb.staged_predict_proba(Xval)]
mejor_n_arboles = int(np.argmax(auc_val_por_arbol)) + 1         # 12
caida_final = max(auc_val_por_arbol) - auc_val_por_arbol[-1]    # 0.039
```

12

mejor nº de árboles (AUC val. 0,741)

0,039

caída del AUC si seguimos hasta 300

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer el mejor número de árboles (ejercicio 1)

staged_predict_proba

Da las probabilidades **tras cada árbol añadido**: así se ve cómo evoluciona el AUC.

np.argmax(...) + 1

argmax da una **posición** (desde 0); el nº de árboles empieza en 1: la posición 11 es el árbol 12.

Entrenamiento sube, validación no

Entrenamiento llega a ≈ 0,935; validación sube hasta **0,741 con 12 árboles** y baja a 0,702: **sobreajuste**.

Qué pierdes

Seguir hasta 300: **0,039 de AUC** en validación, más tiempo de cálculo: un modelo peor y más caro.

CONSULTA · EJERCICIO 2

XGBoost con parada temprana (ejercicio 2)

```
modelo_xgb = xgb.XGBClassifier(n_estimators=1000, learning_rate=0.05, max_depth=3, subsample=0.8,
                               colsample_bytree=0.8, early_stopping_rounds=30, eval_metric="auc",
                               random_state=SEMILLA, n_jobs=2)
modelo_xgb.fit(Xfit, yfit, eval_set=[(Xval, yval)], verbose=False)
arboles_xgb = int(modelo_xgb.best_iteration)                                    # 69
auc_test_xgb = roc_auc_score(y_test, modelo_xgb.predict_proba(Xte)[:, 1])    # 0.745
```

69

árboles en que paró

0,745

AUC en test

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer XGBoost (ejercicio 2)

eval_set=[(Xval, yval)]

Es la **validación** que vigila: sin ella no hay parada y entrena los 1000 árboles.

early_stopping_rounds=30

Para si la validación **no mejora en 30 rondas** seguidas; **best_iteration** es la mejor ronda.

Por qué no el test

La validación **decide cuándo parar**; el test queda **intacto** para evaluar al final con honestidad.

69 frente a 12 árboles

Con **learning_rate** **0,05** (en vez de 0,2) hacen falta más árboles, pero el resultado es más fino: **0,745** en test.

CONSULTA · EJERCICIO 3 (CASA)

LightGBM con parada temprana (ejercicio 3)

```
modelo_lgb = lgb.LGBMClassifier(n_estimators=1000, learning_rate=0.05, num_leaves=8, subsample=0.8,
                                subsample_freq=1, colsample_bytree=0.8, random_state=SEMILLA, n_jobs=2, verbose=-1)
modelo_lgb.fit(Xfit, yfit, eval_set=[(Xval, yval)], eval_metric="auc",
               callbacks=[lgb.early_stopping(30, verbose=False)])
arboles_lgb = int(modelo_lgb.best_iteration_)                                  # 49
auc_test_lgb = roc_auc_score(y_test, modelo_lgb.predict_proba(Xte)[:, 1])    # 0.745
```

49

árboles en que paró

0,745

AUC en test

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer LightGBM (ejercicio 3, casa)

num_leaves=8

Limita el **tamaño de cada árbol** por hojas, no por niveles: LightGBM crece por la hoja que más mejora.

callbacks=[lgb.early_stopping(30)]

Aquí la parada va en un **callback**; sin él (o sin **eval_set**) entrena los 1000 árboles.

best_iteration_

En LightGBM lleva **guion bajo** al final; en XGBoost no (**best_iteration**).

Mismo AUC que XGBoost

0,745 en los dos; cambia el nº de árboles (49 frente a 69) porque son árboles de distinto tipo.

CONSULTA · EJEMPLO (CASA)

Importancia de variables de XGBoost

```
imp = pd.Series(modelo_xgb.feature_importances_, index=nombres).sort_values(ascending=False).head(8)    # top 8
```

0,083

deposito_sin_deposito, la primera

0,082

antelacion_dias, muy cerca

La importancia está **repartida**: varias variables con valores parecidos (≈ 0,04–0,08).

CONSULTA · ACLARACIONES · IMPORTANCIA DE VARIABLES

Leer la importancia de XGBoost (casa)

feature_importances_

Ganancia relativa de cada columna **sumada en todos los árboles**.

.head(8)

Se quedan las **8** más importantes de la Serie ya ordenada.

Repartida entre varias

Con cientos de árboles, la importancia se reparte: depósito, canal, tarifa y antelación (casi todas **columnas one-hot**).

Describe el modelo

Dice qué columnas **usó** el modelo, no si cambiarlas **provoca** o evita la cancelación.

CONSULTA · MINI-RETO (CASA)

Tres implementaciones (mini-reto)

```
tabla = pd.DataFrame({
    "sklearn_gb": {"auc_test": auc_sk, "arboles": modelo_sk.n_estimators_, "segundos_ajuste": t_sk},
    "xgboost": {"auc_test": auc_test_xgb, "arboles": arboles_xgb, "segundos_ajuste": t_xgb},
    "lightgbm": {"auc_test": auc_test_lgb, "arboles": arboles_lgb, "segundos_ajuste": t_lgb}}).T.round(3)
```

Implementación	AUC en test	Árboles	Ajuste (orientativo)
sklearn_gb	0,744	139	≈ 0,7 s
xgboost	0,745	69	≈ 0,1 s
lightgbm	0,745	49	≈ 0,1 s

AUC casi idéntico: decide el **coste** (tiempo, mantenimiento, explicabilidad).

CONSULTA · ACLARACIONES · MINI-RETO

Leer la comparativa (mini-reto, casa)

time.perf_counter()

Se llama **antes y después** de cada **.fit** y se resta: son los **segundos de ajuste**.

n_estimators_

En scikit-learn da el **nº de árboles** que quedaron; en XGBoost y LightGBM, **best_iteration**.

Los tiempos cambian

Varían mucho con el ordenador y la ejecución; la primera vez, XGBoost puede tardar más por el **arranque**.

Qué elegirías

El AUC es el mismo (diferencias de **0,001**), así que decide el coste: **LightGBM** o XGBoost por velocidad.